FUNDAMENTOS DE COMPUTADORES · EXAMEN

Guía del examen P1–P5

Cómo es el examen, cómo se puntúa y por dónde empezar a estudiar, y el método de cada tipo de problema con problemas de convocatorias anteriores resueltos. Intenta cada problema en papel antes de abrir la resolución y márcalo como hecho.

1

Antes de empezar

Cómo es el examen

Estructura, puntuación, cómo tratar la penalización del test y por dónde empezar a estudiar.

1.1 Estructura y puntuación

El examen tiene dos partes y una tercera opcional. La nota del examen es el **90 %** de la final (el 100 % si no hay evaluación continua) y hay que llegar a **5** para aprobar.

Parte	Puntos	Qué es
Test	4	40 preguntas de tres opciones (a/b/c), 0,1 puntos cada una.
Problemas	5	Normalmente tres: **P1** volcado de okteta (C2 e IEEE 754), **P2** circuitos, Linux o redes, y **P3** ensamblador x86-64 con GDB.
P4	1	Sólo para quien no tenga el punto de las tareas entregables.

Cuándo

Convocatoria de enero: **13 de enero de 2027, por la tarde**. La cuenta atrás está en la portada de FC.

1.2 La penalización del test

Cada 2 incorrectas anulan 1 correcta; las que dejas en blanco ni suman ni restan. Es decir, cada fallo vale −0,05 puntos.

Situación	Valor esperado de contestar	Qué hacer
No descartas ninguna opción	$1/3 \cdot 0{,}1 - 2/3 \cdot 0{,}05 = \mathbf{0}$	En blanco: arriesgas sin ganar nada de media.
Descartas una opción	$1/2 \cdot 0{,}1 - 1/2 \cdot 0{,}05 = \mathbf{+0{,}025}$	Contesta: juegas a favor.
Dudas entre dos pero una te convence más	más de +0,025	Contesta.

> ⚠ **Regla práctica**
>
> Contesta sólo cuando puedas descartar al menos una opción. El simulacro aplica exactamente esta penalización para que te acostumbres.

Hacer un simulacro con reloj →

1.3 Por dónde empezar

El orden no es el del temario sino el de rentabilidad: primero lo que más puntos da y más se repite.

1. **Tema 2 · Representación de la información.** Es el problema P1 entero (2 puntos) y entre 7 y 10 preguntas del test. Si sólo tuvieras tiempo para un tema, sería este.
2. **Tema 5 · Lenguajes del computador.** El problema de GDB cae en todas las convocatorias y es muy mecánico en cuanto entiendes el método.
3. **Tema 3 · Circuitos.** Karnaugh, multiplexores y retardos: muchas preguntas de test y a menudo un problema.
4. **Tema 4 · Linux.** Memorizar comandos da puntos rápidos; los tienes todos en la chuleta.
5. **Tema 6 · Redes.** Las subredes son puro procedimiento: se dominan en una tarde.
6. **Tema 1 · Introducción.** Cuatro fórmulas y algo de historia. Déjalo para el final.

Para cada tema: lee el apartado en la guía de teoría (y márcalo como estudiado), haz su autotest y, cuando falles, lee la explicación: te dice qué repasar. Las herramientas son para comprobar ejercicios hechos a mano, nunca para resolverlos.

1.4 El día antes

1. Haz un simulacro completo con reloj y mira el desglose por temas.
2. Repasa sólo los dos temas con peor resultado.
3. Rehaz en papel un P1 de okteta y un problema de GDB de principio a fin (los tienes abajo).
4. Lee la chuleta entera una vez.

2

Problema · 2 puntos · tema 2

P1 · Volcado de okteta

Enteros en complemento a 2 y reales IEEE 754 en un volcado hexadecimal: el problema más mecánico y el que más vale.

2.1 Método

Siempre es igual: un volcado de okteta con varios datos seguidos; hay que decodificar unos, codificar otros e indicar **los desplazamientos de inicio y fin** de cada uno. Vale 2 puntos y es el más mecánico de todos.

1. **Dibuja el mapa de bytes.** Tamaño de cada dato en orden: entero de 32 bits = 4 bytes, de 16 = 2, real simple = 4, doble = 8, cadena = tantos bytes como caracteres.
2. **Sitúa los desplazamientos.** Empieza en 0x0000 y ve sumando. El final de un dato es **inicio + tamaño − 1** (los dos incluidos): aquí se pierden muchos puntos por un byte.
3. **Mira el convenio.** En little endian, **invierte los bytes** de cada dato numérico antes de operar; en big endian se leen tal cual. Las cadenas **nunca** se invierten.
4. **Aplica la receta** de C2 o de IEEE 754 (guía de teoría, apartados 2.3 y 2.5).
5. **Escribe todos los pasos.** El enunciado dice que el apartado sólo es correcto si figuran: el procedimiento vale más que el resultado.

Para comprobar tus cuentas: el mini-okteta y la calculadora de C2 con pasos.

2.2 Problemas resueltos

Problema 1 · Junio 2026 (little endian) JUNIO 2026

Volcado

```
0000:0000 46 46 2E 20 43 6F 6D 70 75 74 61 64 6F 72 65 73 20 6A 75  |FF. Computadores ju|
0000:TTTT 6E 69 6F 20 32 30 32 36 00 00 8C 41 XX XX XX XX XX XX XX  |nio 2026...A.......|
0000:0026 XX 10 B6 FD FF YY YY
```

Contenido, por orden: 1) cadena ASCII, 2) real simple, 3) real doble (oculto con XX), 4) entero de 32 bits en C2, 5) entero de 16 bits en C2 (oculto con YY).

a) ¿Cuánto vale TTTT? **b)** Valor del real simple y sus desplazamientos. **c)** Codifica 31,5 en los XX. **d)** Valor del entero de 32 bits. **e)** Codifica 15 000 en los YY.

---

a) La primera fila tiene 19 bytes (cuéntalos), así que la segunda empieza en 19 = **0x0013**.

b) «FF. Computadores junio 2026» ocupa 27 bytes, de 0x0000 a 0x001A. El real simple va de **0x001B a 0x001E**: `00 00 8C 41` → invertido `41 8C 00 00` = 0100 0001 1000 1100 0…

s = 0; e = 1000 0011 = 131 → E = 131 − 127 = 4; M = 1,00011 → $1{,}00011 \times 2^4 = 10001{,}1_2 = 16 + 1 + 0{,}5 =$ **17,5**.

c) $31{,}5 = 11111{,}1_2 = 1{,}11111 \times 2^4$ → E = 4, e = 4 + 1023 = 1027 = $100\ 0000\ 0011_2$, s = 0, m = 11111 y ceros hasta 52 bits.

```
0 10000000011 1111100…0 = 0100 0000 0011 1111 1000 0…
= 40 3F 80 00 00 00 00 00
little endian → 00 00 00 00 00 80 3F 40
```

d) Va de 0x0027 a 0x002A: `10 B6 FD FF` → invertido `FF FD B6 10` = 1111 1111 1111 1101 1011 0110 0001 0000. El primer bit es 1: negativo. Invierto y sumo 1 → 0000 0000 0000 0010 0100 1001 1111 0000 = $2^{17} + 2^{14} + 2^{11} + 2^8 + 2^7 + 2^6 + 2^5 + 2^4 = 150\ 000$ → **−150 000**.

e) Positivo, no se complementa: 15 000 = 0011 1010 1001 1000_2 = `3A 98` → little endian **`98 3A`**.

Problema 2 · **Enero 2023 (little endian)** ENERO 2023

Volcado

```
0000:0000 45 78 61 6D 65 6E 20 46 43 14 05 00 00 00 00 70  |Examen
FC......p|
0000:0010 3F YY YY YY YY XX XX XX XX                       |?........|
```

Por orden: 1) cadena; 2) entero de 32 bits en C2; 3) real simple; 4) real simple oculto (YY); 5) entero de 32 bits oculto (XX).

a) Tamaño de la cadena. **b)** Valor del entero y sus desplazamientos. **c)** Valor del real. **d)** Codifica −280,25 en los YY. **e)** Codifica −2023 en los XX.

---

a) «Examen FC» son **9 bytes**, de 0x0 a 0x8.

b) De **0x9 a 0xC**: `14 05 00 00` → `00 00 05 14`. Bit de signo 0: positivo. $2^2 + 2^4 + 2^8 + 2^{10} = 4 + 16 + 256 + 1024 =$ **1300**.

c) De 0xD a 0x10: `00 00 70 3F` → `3F 70 00 00` = 0011 1111 0111 0000... s = 0; e = 0111 1110 = 126 → E = −1; M = 1,111. N = $1{,}111 \times 2^{-1} = 0{,}1111_2 =$ **0,9375**.

d) $280{,}25 = 100011000{,}01_2 = 1{,}0001100001 \times 2^8$ → E = 8, e = 135 = 1000 0111, s = 1.

```
1 10000111 00011000010000000000000 = C3 8C 20 00
little endian → 00 20 8C C3
```

e) 2023 = 0000 ... 0111 1110 0111 → invertir → 1111 ... 1000 0001 1000 → +1 → `FF FF F8 19` → little endian **`19 F8 FF FF`**.

⚠ Dos erratas de las soluciones oficiales

En el apartado d) de junio 2026 la solución oficial escribe «−150000» pero la descomposición que da suma 150 000 sin signo; y en mayo de 2024 el enunciado pide codificar 28,625 mientras la solución desarrolla 28,125. Si al corregirte algo no cuadra, comprueba si el fallo es tuyo o suyo.

3

Problema · 1,5–2 puntos · tema 3

Circuitos combinacionales

Karnaugh, retardo del camino crítico e implementación con multiplexor.

3.1 Problema resuelto

Problema 3 · Simplificación por unos y multiplexor ENERO 2026

S(A,B,C) vale 1 en 000, 001, 100, 101 y 111, y 0 en el resto.

a) Suma de minitérminos. **b)** Simplifica por unos con Karnaugh. **c)** Dibuja el circuito y calcula su retardo (NOT 2 ns, AND 5 ns, OR 6 ns). **d)** Impleméntalo con un multiplexor controlado por B y C.

---

a) S = A'B'C' + A'B'C + AB'C' + AB'C + ABC

b) Las cuatro celdas con B = 0 forman un grupo (se eliminan A y C) → **B'**. 101 con 111 forman otro (se elimina B) → **AC**. **S = B' + A·C**.

c) Dos caminos: B → NOT (2) → OR (6) = 8 ns; A, C → AND (5) → OR (6) = 11 ns. El retardo es el del camino crítico: **11 ns** (los caminos en paralelo no se suman).

d) Control en B y C → MUX 4×1 con A como variable excluida. Para cada par (A = 0, A = 1):

BC	S con A = 0	S con A = 1	Entrada del MUX
00	1	1	**1**
01	1	1	**1**
10	0	0	**0**
11	0	1	**A**

Compruébalo con el mapa de Karnaugh interactivo.

3.2 Recetas y errores

Receta del MUX con una variable excluida

Agrupa el mapa por parejas que sólo difieran en la variable excluida y, para cada entrada del MUX:

F con X = 0	F con X = 1	Entrada
0	0	0
1	1	1
0	1	X
1	0	X'

El número de la entrada lo forman las variables de control en el orden del enunciado: con control A, B, C, la entrada 5 es A = 1, B = 0, C = 1.

Errores típicos

- Sumar los retardos de dos ramas en paralelo, u olvidar el NOT del camino crítico.
- Para «sólo NAND» hay que simplificar **por unos**; para «sólo NOR», **por ceros**.

4

Problema · 1,5 puntos · tema 4

Linux

Escribir órdenes con rutas relativas y reconstruir las que faltan en una sesión.

4.1 Problemas resueltos

Problema 4 · **Escribir las órdenes** ENERO 2023

Situación

```
[belen@tyche docencia]$ ls -R
.:
copia_fc.tgz   fc   etc
./fc:
pruebas.sh  planning.pdf  datos.odt  teoria
./fc/teoria:
apuntes01.txt  apuntes02.txt  apuntes03.txt
[belen@tyche docencia]$ cd fc
```

Estás en `~/docencia/fc`. Escribe, con rutas relativas, la orden para:

1. Ver `apuntes01.txt` pudiendo avanzar y retroceder.
2. Listado detallado de `etc` (cuelga de docencia), con ocultos.
3. Buscar desde el directorio padre los ficheros regulares de más de 4 KB acabados en «.txt».
4. Extraer `copia_fc.tgz`.
5. Ver el espacio usado y libre del sistema de ficheros.
6. Listar todos los procesos del sistema.
7. Mostrar ordenado el contenido de los ficheros que empiezan por p, q, r o s con extensión de tres caracteres, añadiendo los errores a errores.txt.
8. Saber tu nombre de usuario.

```
less teoria/apuntes01.txt
ls -la ../etc
find .. -type f -name "*.txt" -size +4k
tar xzvf ../copia_fc.tgz
df -h
ps -Af
sort [p-s]*.??? 2>> errores.txt
whoami
```

Lo que puntúa: las comillas en `-name`, el `..` de las rutas relativas y la diferencia entre `2>` (machaca) y `2>>` (añade).

Problema 5 · Reconstruir las órdenes JUNIO

Sesión

```
maria@lenovo:~$ ###ORDEN 1###
/home/maria
maria@lenovo:~$ mkdir publico
maria@lenovo:~$ ls -ld publico
drwxrwxr-x 2 maria maria 4096 publico
maria@lenovo:~$ ###ORDEN 2###
maria@lenovo:~$ ls -ld publico
drwxrwxrwx 2 maria maria 4096 publico
```

¿Qué órdenes son la 1 y la 2? ¿Qué consecuencia tiene la 2? Escribe también cómo empaquetar en `examenes.tgz` los ficheros `examen_fc_*`, cómo recuperarlos y borrar el .tgz en una línea, el tamaño total del directorio en una línea y el listado de procesos con UID, PID, PPID y STIME.

- **Orden 1:** imprime la ruta absoluta del directorio actual → `pwd`.
- **Orden 2:** los permisos pasan de 775 a 777 → `chmod 777 publico`. Cualquier usuario puede listar, crear y borrar dentro y entrar.
- Empaquetar: `tar czf examenes.tgz examen_fc_*`
- Recuperar y borrar: `tar xzf examenes.tgz ; rm examenes.tgz`
- Tamaño total: `du -hs`
- Procesos: `ps -Af`

Estrategia: no intentes adivinar la orden de golpe; mira qué ha cambiado entre antes y después y qué columnas salen.

4.2 Lo que más se confunde

⚠ Lo que más se confunde

- `find -mtime -2` = modificados hace **menos** de 2 días («en los últimos dos días»).
- `>` machaca, `>>` añade; `2>` sólo redirige los errores.
- Permisos de directorio: r = listar, w = crear y borrar dentro, x = entrar.
- Ctrl-C mata y Ctrl-Z duerme el proceso en primer plano; para uno en segundo plano o colgado, `kill -9 PID`.

Calculadora de permisos: chmod ↔ rwx.

5

Problema · 1,5–2,4 puntos · tema 5

GDB y ensamblador

Leer un desensamblado y una sesión de GDB: cae en todas las convocatorias.

5.1 Las siete preguntas de siempre

Te dan un programa en C, su desensamblado con direcciones y una sesión de GDB con registros y un volcado de memoria. Las preguntas son siempre de esta familia:

1. **¿Dónde está detenido?** En la dirección de **RIP**: es la *siguiente* instrucción, todavía no ejecutada.
2. **¿Cuánto ocupa una instrucción?** Dirección de la siguiente menos la suya: de `<main+1>` a `<main+6>` son 5 bytes.
3. **¿Cuánto ocupa una subrutina?** Inicio de la siguiente subrutina menos el suyo.
4. **¿Su código máquina?** Del volcado `x/Nbx`, contando bytes desde el inicio según el desplazamiento `<nombre+k>`.
5. **¿Qué vale v[i]?** Dirección = base + i × tamaño (4 si es `int`, 1 si es `char`), y los bytes del volcado en **little endian**.
6. **¿Dónde está una variable local?** En `RBP - desplazamiento`: si `i` está en `-0x4(%rbp)` con RBP = 0x7fffffffdc80, ocupa de 0x7fffffffdc7c a 0x7fffffffdc7f.
7. **¿En qué iteración va?** Mira el registro índice: si vale 7, se han completado 7 vueltas y va por la octava.

Código C	Ensamblador
`i = 0;`	`movl $0x0,-0x4(%rbp)`
`i++;`	`addl $0x1,-0x4(%rbp)`
`while (i < N)`	`cmpl $0xa,-0x4(%rbp)` + `jl …`
`v[i]`	`mov base(,%rax,4),%edx`
`return x;`	el valor en `%eax` y `ret`
`f(x)`	parámetro en `%edi` y `call`

5.2 Problemas resueltos

Problema 6 · La nota media de una clase

C

```
#define N 10
int notas[N] = {5,6,7,8,5,6,7,4,5,6};
int media = 0;
int main() {
  int i = 0;  int suma = 0;
  while (i < N) { suma = suma + notas[i]; i++; }
  media = suma / N;
}
```

Desensamblado y GDB

```
0x400508 <main+8>   movl $0x0,-0x4(%rbp)
0x40050f <main+15>  movl $0x0,-0x8(%rbp)
0x400516 <main+22>  jmp  0x40052c
0x400518 <main+24>  mov  -0x4(%rbp),%eax
0x40051b <main+27>  cltq
0x40051d <main+29>  mov  0x601040(,%rax,4),%edx
0x400524 <main+36>  add  %edx,-0x8(%rbp)
0x400528 <main+40>  addl $0x1,-0x4(%rbp)
0x40052c <main+44>  cmpl $0xa,-0x4(%rbp)
0x400530 <main+48>  jl   0x400518
0x400536 <main+54>  idivl …
0x400539 <main+57>  mov  %eax,0x601070

(gdb) info registers rip rax rbp
rip 0x400524   rax 0x7   rbp 0x7fffffffdc80
```

a) ¿Qué instrucción inicializa `suma`? **b)** ¿Entre qué direcciones está `i`? **c)** ¿Qué instrucción accede a `notas[i]`? **d)** ¿A qué dirección accede ahora? **e)** ¿Qué valor lee? **f)** ¿Cuántas veces se ejecuta la de 0x400524? **g)** ¿Dónde se guarda `media`?

---

a) `movl $0x0,-0x8(%rbp)`: es la segunda inicialización a 0 y `-0x8(%rbp)` es donde luego se acumula.

b) En `-0x4(%rbp)`: de 0x7fffffffdc80 − 4 = **0x7fffffffdc7c** a **0x7fffffffdc7f** (un `int`, 4 bytes).

c) `mov 0x601040(,%rax,4),%edx`: 0x601040 es la base del array, RAX el índice y 4 el tamaño de un `int`.

d) RAX = 7 → 0x601040 + 7 × 4 = 0x601040 + 0x1C = **0x60105C**.

e) En 0x60105C el volcado tiene `04 00 00 00` → little endian → **4**, que es `notas[7]`.

f) El bucle va de i = 0 a 9 → **10 veces**.

g) En **0x601070** (`mov %eax,0x601070`): es global, con dirección fija en el segmento de datos.

Problema 7 · Programa en ensamblador x86-64 JUNIO 2025

GDB

```
rax 0x404028     rbx 0xYY

0x40111d <main>      mov  $0x404028,%eax
0x401122 <main+5>    mov  $0x0,%ebx
0x401127 <main+10>   mov  (%eax),%edi
0x40112a <loop>      cmp  $0xa,%edi
0x40112d <loop+3>    je   0x401137 <fin>
0x40112f <loop+5>    add  $0x1,%ebx
0x401132 <loop+8>    add  $0x5,%edi
0x401135 <loop+11>   jmp  0x40112a <loop>
0x401137 <fin>       mov  %edi,(%eax)

(gdb) x/8bx 0x404028
0x404028:  0x05 0x00 0x00 0x00 0x00 0x00 0x00 0x00
```

La última instrucción ejecutada está en 0x401122. **a)** ¿Qué hizo? **b)** ¿Cuánto vale RBX y de dónde sale? **c)** ¿Qué valdrá EDI al llegar a 0x40112a? **d)** ¿Cuántos saltos incondicionales se ejecutarán? **e)** ¿Qué instrucción escribe en memoria y dónde?

---

a) `mov $0x0,%ebx`: movimiento de datos que escribe la constante 0 en **EBX** (32 bits). Su operando de entrada es un **inmediato**.

b) **0**. Sale de una constante codificada en la propia instrucción, es decir, del **segmento de código**.

c) Antes se ejecuta `mov (%eax),%edi`: lee 4 bytes desde 0x404028 → `05 00 00 00` → **5** (ocupa de 0x404028 a 0x40402B).

d) **Uno.** Con EDI = 5 el `je` no salta, se suma 5 y se llega al `jmp`. En la segunda vuelta EDI = 10, el `je` salta a `fin` y el `jmp` ya no se ejecuta.

e) `mov %edi,(%eax)` en `fin`: escribe EDI (que vale 10 al salir) en **0x404028**, porque EAX no cambia en todo el programa.

6

Problema · 1,2–1,5 puntos · tema 6

Redes

Leer la configuración de un equipo y calcular subredes.

6.1 Receta de subredes

Receta de subredes

1. Bits de host h = 32 − n (si te dan la máscara, pasa a binario el octeto que no sea 0 ni 255: 240 = 11110000 → 4 unos).
2. Red: bits de host a 0. Broadcast: bits de host a 1.
3. Primer host = red + 1; último = broadcast − 1.
4. Hosts posibles = $2^h - 2$.

6.2 Problemas resueltos

Problema 8 · **Análisis de una configuración de red** JUNIO 2026

Terminal

```
user@host:~$ ifconfig
eth0  direcciónHW 00:1d:7d:ab:0d:3e
      Direc. inet:155.54.205.34  Másc:255.255.254.0
user@host:~$ nmcli
DNS configuration: servers: 155.54.1.10 155.54.1.1 155.54.1.2
user@host:~$ host www.wikipedia.com
www.wikipedia.com has address 91.198.174.192
```

a) ¿Qué otra IP se refiere siempre a este host? **b)** ¿Cuántas tarjetas de red caben como máximo en su subred? **c)** Identifica la subred y su broadcast. **d)** ¿Qué comando da la IP del router? ¿Podría ser 155.54.205.0? **e)** ¿Qué protocolo usa `host`?

---

a) **127.0.0.1** (localhost), disponible incluso sin red.

b) 255.255.254.0 deja 9 bits de host → $2^9 - 2 =$ **510**.

c) /23 → red **155.54.204.0/23**, broadcast **155.54.205.255** (205 = 11001101: con el bit de host a 0 queda 204).

d) `route -n` (la pasarela de la fila con destino 0.0.0.0). Y **sí** podría ser 155.54.205.0: está dentro de la subred y no es ni la dirección de red ni la de broadcast. Acabar en .0 no la invalida si la máscara no es /24.

e) **DNS**, que traduce nombres a IP y al revés; consulta a cualquiera de los servidores de `nmcli`.

Problema 9 · Dividir una /26 en dos subredes de 16 PC

Partiendo de 155.54.200.0/26, haz dos subredes con 16 equipos cada una usando el mínimo rango. Da máscara, red, broadcast, rango válido y router (primera IP válida) de cada una.

1. 16 PC + router = 17 direcciones de host, más red y broadcast. Con 4 bits de host sólo hay 14 válidas: hacen falta **5 bits** (30 válidas) → **/27 = 255.255.255.224**.
2. **Subred 1:** red 155.54.200.0, broadcast 155.54.200.31, válidas .1 – .30, router 155.54.200.1.
3. **Subred 2:** red 155.54.200.32, broadcast 155.54.200.63, válidas .33 – .62, router 155.54.200.33.

Compruébalo en la calculadora de red.

Repaso

Errores que más puntos cuestan

Revísalos la víspera.

7.1 La lista

- Olvidar invertir los bytes en **little endian**, o invertirlos en una cadena (las cadenas nunca se invierten).
- Equivocarse en un byte en el **desplazamiento final**: es inicio + tamaño − 1.
- Confundir e con E en IEEE 754 (E = e − 127 en simple, e − 1023 en doble).
- Poner `-mtime +2` cuando piden «los últimos dos días» (es `-2`).
- Sumar los retardos de ramas que van en **paralelo**.
- Confundir codificador ($2^n \rightarrow n$) con decodificador ($n \rightarrow 2^n$).
- Olvidar restar 2 en $2^h - 2$ al contar hosts.
- Decir que RIP apunta a la instrucción que se acaba de ejecutar: apunta a la **siguiente**.
- Leer «5 bits de campo de **registro**» como operaciones, o al revés.
- Contestar al azar en el test: el valor esperado es cero o negativo.

Elaborado a partir de los exámenes resueltos de las convocatorias de 2022 a 2026.